

Rubicon principles

Work in progress. This page is mirrored automatically from [rubicon/docs/principles.md](#) in the Causal Map repository and changes as the design does. The section written for whoever changes the code is not published here.

This should matter to you if:

- you are an **evaluator** using or planning to use AI for qualitative processing of texts and you want to be able to show that your work is of high methodological quality, you avoided all the pitfalls and your work could, more or less, be reconstructed by an independent adjudicator
- you are an evaluation **commissioner** who wants to make sure your evaluator(s) use robust approaches for qualitative text processing, especially when using AI

- you are an **academic or thought leader** interested in evaluation methods and how they are, and should be, applied.

Here we suggest an open format for describing qualitative text-processing workflows in evaluation. Further down we also describe software, Rubicon, which already implements it; but because the format is open, anyone could build software to do the same thing.

The problem

AI has arrived in evaluation for qualitative processing of texts, and it is staying.

Qualitative text processing is a key part of most evaluation: gathering and reading interviews and reports to answer questions which are often poorly defined. Some approaches (also) use quantitative methods, but even these are often embedded in, or have to combine with broader, qualitative work.

Until recently, the question has often been: are you using AI? Are you allowed to? Can evaluation commissioners tell if AI has been used?

Whereas now in late 2026, AI has changed the market and the profession. Just about anyone who wants to work competitively is using AI, and if they aren't now, they will be soon. And the spectre of hallucinated results and mitigating calls for a "human in the loop" has a reverse side: given the *potential* for AI when used well, why are you *not* using AI? A human checker might find errors in AI processing, but you can bet your bottom dollar on the reverse bet: that AI will find at least as many errors if asked to check *human* work.

Used well, AI raises quality on narrative material, more cheaply than anything before. Used badly it produces junk — even faster. In the final evaluation report the two are impossible to tell apart: both sound confident and fluent, both quote passages that fit. What evaluation commissioners and evaluators should be asking is:

what is the difference between a robust workflow and a junk one, and how can we ensure that evaluators use robust workflows?

What not to do with AI:

- Don't let an AI (or a human) make global judgements without clear procedures or criteria. Don't ask an AI to "summarise this document" or "what are the main themes here" or worst "is the project effective, according to the sources?"

So do not ask an AI (or a human) for a direct answer to a vague question. It will answer confidently, having decided for itself what you meant.

Vague questions are great! But...

Human progress is based on asking vague questions. But answering them robustly is all about *coming to an agreement about how to answer them*. Humans at least, and good AIs should push back on a direct question: "I can see where you are going with this, but as it stands, this question is not answerable. Let me help you break this question down so it is answerable; maybe you need to check back with your stakeholders, so this might take a while."

What to do: Break down a vague question into **well-defined ones**.

A well-defined question is one that two people with graduate-level generic social science skills would answer much the same way from the same material. The word is borrowed from the problem-solving literature, where a problem is ill-defined when the statement leaves parts of it open, so that no answer will be universally accepted (Reitman 1964), and where well and ill defined are the ends of a continuum rather than a binary choice (Simon 1973). We call a broken-down method for answering a question a **workflow**. It might be as simple as "count the instances of the word 'health' in each report and add them all up." But most interesting questions need more sophisticated workflows, perhaps splitting the task up into separate pieces, one for each subquestion, one for each source, perhaps iterating and looping and rejoining, and finally making an **evaluative judgement** about the value, significance or worth of the finding.

At Causal Map, we have long been proponents of a version of this "breaking down" strategy especially when using AI: see [garden.causalmap.app](#). But that approach involves just one specific well-defined task: identifying causal claims. That fits perfectly into this kind of format. But it is of course not the only such task.

But wait, even humans don't yet agree how best to break down evaluation questions!

That's true. Many specific approaches like Outcome Harvesting or Process Tracing have manuals on how to apply them, but in practice most evaluators use, and need to use, varying kinds of bricolage to mix and adapt the methods to their needs (Aston & Apgar 2022; Apgar et al. 2024; Apgar & Aston 2025). But that does not mean that "anything goes".

So: an open format for qualitative text processing workflows in evaluation

Let's look at a way to specify workflows for breaking down one vague question into multiple, more clearly answerable ones.

Most evaluators are probably using Claude or ChatGPT right now, maybe in quite sophisticated ways they thought up themselves. Perhaps they keep track of decisions they make and the reasons for them, perhaps not. Who knows? Perhaps they used clever techniques to force their assistant to identify negative cases and address satisficing, perhaps not. Who knows? The reader of an ordinary evaluation report cannot see. Which passages were counted and which were passed over; what rule set the threshold at "more than half", how much agreement is needed to count as "many" or "substantially"; whether that rule was written before or after the first results came through; whether a second analyst would have even identified the same passages.

AI makes this problem worse, because a model reads a thousand pages overnight and nobody can check its working. But AI also makes the problem more easily fixable, because a machine that reads can be made to show every step.

This kind of format is agnostic as to whether humans or AI are following it. We call it **machine-readable** (like post-codes on envelopes) not because it has to be read by a machine, but because it is clear enough that even a machine could follow it, more or less.

There is no such format yet, and the nearest, REFI-QDA, standardises the coded result rather than the procedure that produced it. See this note: [Is there already a standard?](#)

Participation and reflection built-in

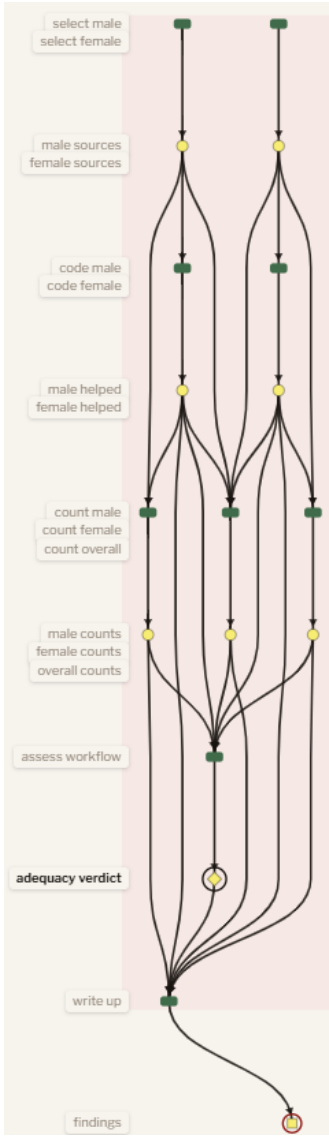
Participation (involving other stakeholders) and reflection (on the part of the analyst) are not add-ons to many evaluation workflows such as Outcome Harvesting; they are the most important part. But again, that does not mean "anything goes". An open format can set out a workflow with clearly marked phases which are *outside* the scope of any algorithm, where we just have to pause for participation and/or reflection to complete, before we can continue with a new result — or stop and start all over if that is the conclusion.

A workflow answers a single question; most evaluation tasks have many

Of course most evaluation tasks involve answering multiple, possibly inter-related, questions, not just one. In that case we just use multiple, possibly inter-related workflows. Here we will just focus on individual workflows.

High-stakes tasks often focus just one one question.

A workflow is like a flow diagram, but also a piece of text



One workflow might fan out twelve tests from a single intermediate data set and synthesise the results numerically or in text; split a corpus so the half that wrote a test never scores it; run one instruction twice over ten sources to measure how much the answer depends on the wording rather than on the material; produce several tables from the same coded rows, each with its own base; tabulate a judgement per test before any roll-up; and combine those into one verdict by a rule stated in advance, for example, considering the weakest link rather than the average.

A workflow can be expressed in different ways:

- as a flow diagram or "graph".
- in a special somewhat-human-readable text format called YAML
- less formally, in a human-friendly, easy-to-read format

In fact, the graph and the YAML are just different ways of saying the same thing.

What a workflow can express is set out in full further down.

AI assistance makes using such a format possible

Two years ago it would have been mad to suggest that evaluators or commissioners could ever be bothered to express an analysis plan (unless they had the sort of budgets available for RCTs) in terms of any kind of formal language. But an AI assistant can easily read and understand the format specification and help us to use it and explain what is going on - **but in a way we can check when we want to**. And once the format is specified, it's easy to build software to help evaluators use it.

A format has to be extensible

No-one is going to force evaluators or commissioners to a single set of tools. We can agree on the basic shape and extend it formally or informally. We can aim to provide a small, powerful, general set of text-processing tools which can be combined and recombined for different purposes, meaning that evaluators will most often not need to build new tools when they have a new idea, they can just build one out of existing tools, or most likely ask their AI assistant to do that.

Text judgements and rubrics

A step in a workflow might say to produce a text summary of a set of findings. Sometimes that is useful, even if it moves us away from being well defined; ask someone else and you'd get a different answer. Sometimes it helps to specify more clearly a simpler, less controversial kind of summary, like 'write a short version of the main points of each finding'.

Evaluators often use scales to condense a set of texts (and/or numbers) into a **single summary** (one out of a narrow range of options) according to rules expressed clearly in text; a **rubric** is a special kind of scale which attaches a different *value* to the levels of the scale, used for making evaluative judgements: is this level good, or good enough. Attaching value is an act which humans will certainly want to lead.

We do not suggest for a moment the application of pre-determined rubrics, like "**overwhelming** means at least 8 out of 10" or anything like that. The point is to give evaluators a tool for expressing such decisions and making them transparent.

THE STANDARD IT APPLIED	
workflow_adequacy version 2, written by hello@causalmap.app. Agreed by hello@causalmap.app on 2026-09-06 by running what-helped-with-loneliness from Rubicon, where the whole draft including this standard was on the screen.	
coverage: Did enough of the twenty interviews yield at least one coded passage to support a finding?	
• reportable	when interviews, with, any is at least 10 At least ten of the twenty interviews yielded at least one passage – half the base, the minimum for a finding rather than an anecdote.
• partial	when interviews, with, any is at least 5 Between five and nine interviews yielded something – enough to describe what was found but not enough to report shares as representative.
• not_reportable	Fewer than five interviews yielded anything. The coding found too little to report.
gender_reportability: Did both gender strands yield enough passages to support a gender comparison?	
• reportable	when interviews, with, any is at least 6 Both the male strand and the female strand each yielded passages from at least six of their ten interviews.
• partial	when interviews, with, any is at least 0 At least one strand yielded passages from fewer than six of its ten interviews. The gender comparison is not supportable as a finding.
• not_reportable	Neither strand yielded enough to compare.
category_integrity: Do the passages the coding returned actually match the category they were assigned? A passage labelled service, or council should describe a formal or semi-formal provision; one labelled personal, or family should describe a relationship or personal practice; one labelled mixed should genuinely span both.	
• reportable	The passages sampled match their assigned categories. Borderline cases are few and do not change the overall picture.
• partial	Some passages are miscategorised – enough to affect the service/personal split but not enough to invalidate the overall finding.
• not_reportable	The categories are not holding. Passages labelled service, or council describe personal relationships, or vice versa, at a rate that makes the split unreliable.

(Pre)-registration as an option. Restricting freedom.

We asked above whether a rule that, say, set the threshold at "more than half" was written before or after the first results came through. Sometimes we have to, and want to, evolve our method and our criteria as we go along. But with high-stakes summative evaluations sometimes the commissioner might want to restrict the evaluator's freedom to tweak the method or change criteria.

So a format like this can optionally be **registered** with a commissioner, with a version, a date and a named person. What gets registered is the workflow, or as much of it as the commissioner asks for: which sources will be read and how they were drawn, how the text will be coded, what counts as "more than half", and the rubric that turns the counting into a verdict. The rubric is an obvious thing to register, because where "adequate" begins is nowhere in the evidence, and a standard written after the numbers arrive can be made to fit them. But a sampling frame settled after a first look, or a coding instruction narrowed until the finding firmed up, moves the answer just as far, and neither is any more visible in the report.

Registration is a dial rather than a gate. Register nothing and you still hand over a workflow somebody else can read – at inception, and/or later, and/or as it is iteratively developed. Register the sampling and the coding and you have committed to the descriptive half while leaving the verdict open. Register the rubric too and you have said what counts as good before you know what the material says. The commissioner asks for the setting the stakes deserve, and the freedom being given up is the freedom to change your mind without saying so. Everything else stays.

But doesn't that rule out iterating? No. In practice the question turns out not to fit the material, a distinction nobody expected forces new coding columns, a finding rests on one talkative respondent. Iterating means changing the method after seeing something, which is what registration exists to stop. The reconciliation is that **registration records ordering rather than commanding fixity**. Every version is dated and never rewritten, and every run records which version was in force, so a reader can see whether a threshold moved before or after the numbers arrived. That survives any amount of iteration.

So you can:

- choose to not register your workflow at all
- register your workflow but allow logged iterations
- register your workflow and not allow iterations

Software written to support this format should of course, when required, enforce versioning so everyone can see how the plan changed.

Agreeing rubrics before the evidence arrives is sometimes advised in the evaluation literature. Davidson advises drafting them before the evidence is gathered, as advice rather than as a dated record, and pre-registration proper is familiar to evaluators working with controlled trials and experiments. It has also been suggested for evaluation aimed at outcomes and analysis plans, which is the descriptive half (Nosek et al. 2018; Peck and Litwok 2024).

Extensions: Iteration and self-editing

It is not hard in principle to make this kind of format more powerful still, by allowing for loops, so that a workflow can iterate one or more steps a certain number of times or until a criterion is reached. Even more exciting - but arguably not so well-defined - would be to allow one step to edit the instruction of the following step in pre-defined ways, providing something more like an agentic workflow.

Avoiding bias

Iterating on what you have read is how the work gets better, and it is also how a method comes to fit one corpus and nothing else. You read the transcripts, the coding instruction gets narrowed until it finds what you saw, the threshold settles where the numbers happened to fall, and the report presents the result as though the method had been decided independently of it. Nobody has deliberately cheated, and the reader cannot see it either way.

Some mitigations.

Validation with synthetic material

A workflow can run perfectly and still be unable to tell a good case from a bad one. Nothing in its output shows that. We might be very reluctant to preregister or even tell anyone about our plan: what if it does not work?

So before the real material, it is now quite easy to use AI to generate synthetic documents in two sets, one formulated so that they should score well and one that should score badly, then run the initial workflow over both. If it separates the two sets as it should, you have some reason to trust it on the real corpus. If it does not, the problem is the method rather than the material, and you have found that out before it mattered.

Validation with synthetic material is not essential to this suggested format. It is just an example of how having tools to describe workflows accurately (and run them) opens up a whole vista of possibilities which were not possible just two years ago.

Hold material back.

- Develop the workflow on a subset of the material, or analogue material from a previous year or a similar program, then run the settled workflow over the rest.
- Where the workflow writes its own tests, split the corpus so that the half that wrote a test never scores it.

Test the instrument rather than the programme.

Run one instruction twice over the same ten sources and the flip rate is run-to-run variation; run two differently worded instructions over those ten and the difference is instruction sensitivity. Neither is a finding about the world, but both can be informative.

Being aware of absences

- **Search both sides equally.** (AI) coding workflows have to be very careful how to deal with absences and find ways to balance what was found with what was not found. Is seven mentions of depression across a fifty-page interview a lot or a little? Could we compare it with something else?

Anything that can legitimately find nothing must show that it at least looked. A check that passed because it matched nothing, a search pointed at the wrong place, a step that read half of what it was given: all three look exactly like a good result. So a workflow reports what it did as well as what it found, and the reader can tell an empty answer from a question nobody asked.

A negative finding is the hardest case. No quotation can evidence an absence, so demanding one pushes "nobody described a two-way benefit" into citing something irrelevant. A negative finding cites the search instead: what was looked for, where, and how much was read.

A proportion states its base, and the base includes the documents that gave nothing. Build a denominator from the passages already coded and a document that said nothing disappears, so "of the sources that gave me something" reads as "of the sources". On a sparse question, which unintended harm usually is, that inflates the headline figure where it matters most.

The absence an evaluator most often wants is the one no other method gives you: of the things the programme said would happen, which did nobody mention at all.

Rubicon: what it is, and the principles behind it

Rubric and Rubicon share a root, Latin *ruber*, red. A rubric was a heading written in red ochre in the margin of a law book, and the Rubicon was named for the red clay of its bed. Red means take me seriously. Crossing the Rubicon is what this app does. It's up to you to work out which river we are crossing.

What is Rubicon

Rubicon is an experimental wing of the Causal Map code base, with its own page and its own database. If it goes further it becomes a separate product. Its projects are Causal Map projects, so it can read the project's causal map, and each source's map, alongside the documents. But it has its own database and apart from sharing projects and their sources, there is not much connection between them.

It is a way to implement the proposed **open format** for qualitative text processing workflows in evaluation.

The parts of Rubicon

Five things, in the order you meet them.

Projects

You pick a project on the way in. Rubicon writes nothing back to it, so deleting Rubicon tomorrow would leave the Causal Map project exactly as it was.

What you land on is the project hub: every question anybody has asked of this project, what to do next on each, and a standing conversation with Ruby that is not attached to any single question and is shared by everyone on the project.

Sources

The project's documents, whatever was imported into Causal Map: interviews, reports, minutes, field notes. Rubicon reads them and never changes them, and it never treats one as something a workflow produced.

Some of them are the standard rather than the evidence, a theory of change, a funder's standards document, terms of reference. Those are flagged as **background documents**, so a workflow can read one when it needs to know what was promised, and nothing samples, codes or counts them as though somebody had said it in an interview.

Questions

A question is what you want answered, and everything else hangs off one. A project usually has many and they are the unit of work: you ask one, argue out a workflow for it, run that, and accept one result as the answer. Several workflows can attempt the same question, and asking the same thing twice does not make a second question, because every run files itself under the question's own text.

A question is answered exactly when a result stands as its answer, which is read off the answers rather than set by anybody. So nothing can mark a question answered while nothing stands as its answer, and a report can list the questions that have none.

Ruby

The assistant. She reads the corpus, says what she takes a *question* to mean, argues with the parts that will not work, and proposes a *workflow*. She never decides what counts as good.

Rubicon, Causal Map and causal mapping

Rubicon is its own page in Causal Map. It shares the login with Causal Map and reads the project's documents, and past those two things it is not a tab in an existing app.

Rubicon is also not finished, not priced and not yet publicly available, and the working name might change. Working in **Causal Map** is all about creating essentially **one cognitive causal model** of the ideas in the texts. From that model you can extract answers to many evaluation questions; but doing so is sadly not always straightforward. With Rubicon there is no overall model of all the texts. You can answer each question in different way, using a different workflow for each, or choose to build one workflow using the results from other workflows.

- **Ruby is the way into the app** rather than a tool beside it.
- **She learns what the app accepts by being refused.** Refusals are written to teach.
- **She is a methods coach rather than an order-taker.** A question can be perfectly well defined and a poor use of the material: it rests on two speakers out of seventy, or it counts how many said something where the interesting thing is what they said. She says what she would ask instead, then writes the workflow for what you actually asked. The evaluation is yours rather than hers.
- **What gets agreed is the workflow** rather than the conversation.

You state a question and argue out a **workflow** with Ruby, the assistant, before any analysis runs: which sources to read, how to mark them up, how to count what was marked, how to combine the results, how to reach a verdict. Then it runs. What you agreed is the workflow.

The workflow

- **The workflow and optionally the evaluative standard is written down and versioned before the run that applies it reads anything.** Pressing Run is the act of agreeing to it and vouching for this method. If you don't get it, don't do it.
- **Every claim can be traced back to the words in a document.** For example, verdict to figures, figure to rows, row to quote, quote to the span in the source.
- **The evaluative method is written out**, usually as a rubric. The machine runs it and a person checks it.
- **A method can be tested before it meets real material**, on synthetic texts with known verdicts, some good and some bad. Optional, and we believe unique to Rubicon.
- **Ruby builds a workflow out of powerful general ideas** rather than picking a named method off a long menu.
- A workflow answers **one question**. A report gathers the accepted answers of several into one document, a section each, and names the questions that have none.

The workflow canvas

A conversation on one side and a canvas on the other.

- **A workflow is one scrolling document.** Unrun, it is the steps. Run, each result sits in the block of the step that made it. Lineage is the point of the app.
- **A margin map runs beside it**, one viewport tall and sticky, its nodes the steps and results and its edges the

parentage. Click a node to scroll to its block; read a block and its node lights.

- **A step may depend on multiple earlier results or sources, but a result always has just one parent step.** - Click a node and it opens in a **lightbox**: a coding step's wording and the quotes it

found, a judging step's rubric and verdict. A quote opens the source around it.

- **You can walk back from any claim.** A result says which results it came from, a figure opens the rows behind it, and clicking a quote opens its source with the words highlighted.

Some terminology

How they fit together. A *project* holds *sources* and *questions*. For one question you agree a *workflow*, a set of *steps*. Running it makes a *run*, and each step produces a *result*: a *sample*, a *coding*, a *figure*, a *judgement*, a paragraph. Every result records which results it came from, so any claim walks back through *quotes* to the words in a source. When one result answers the question, you *accept* it.

Below, in the order the words turn up in a piece of work. A term in *italics* is defined somewhere else in this list, or under **The parts of Rubicon** above. The rules that go with each term are in the sections after it.

The material

Background document. The flag lives on the *source* itself in Causal Map, in a custom column, so one library serves every *question* you ask of that project. There is no button in the app for it yet; you type it into the column.

Sample. Which *sources* were read, out of which set, chosen how. It is the *result* a **select** or **sample** *step* produces. A draw that leaves anything to chance states the seed it drew on, so the same *workflow* run twice reads the same documents.

Exclusion. A rule over the source columns that drops *sources* from a *sample*, with a reason attached and a count of what was removed.

Unit. What gets counted, and what a *figure* counts across: a person, an organisation, a document. Every threshold needs one, because "more than half of stakeholders" has to know how many there are. Rubicon makes one unit per source and has no other way of making one, so a project whose units are really people, with several interviews to a source, cannot yet say so.

Chunk and segment. A long *source* is cut into equal chunks of at most sixteen thousand characters, one model call each, unless an earlier *step* already narrowed it to passages, when each passage is a call. The whole source is pre-swept into numbered segments and each chunk carries the markers for the ones it holds, so a number means the same thing in every chunk. That numbering is the handle for making a model account for the whole of what it was given rather than stopping at the first good thing.

The question, and agreeing what to do

Well-defined. Two careful readers would apply the *coding* instruction much the same way, and every row leads back to a *quote* a third person could check. Being well defined is what *Ruby* refuses a question for, and nothing else is.

- A property of the question and the material together, usually achieved rather than discovered. Most questions arrive ill-defined, and making them well defined is *Ruby's* job.
- Not the same as objective, factual, measurable or important. A well-defined question can be trivial, and an excellent question can be ill-defined.
- A matter of degree rather than a line. What carries it is the *question*, which is why checking the answer afterwards is a

different thing: an answer can be traceable, checked and sound, and still answer a question nobody asked, because that was the one the reader could see how to answer.

Chat conversation. An exchange with *Ruby* about a *question*. A question has many and a *workflow* has many, neither owning the other. It is a *run* like any other and carries a status, but what it produces is a workflow or nothing, so that status says whether the conversation ran and never whether the workflow is any good.

Rewriting a chat turn. Edit anything you said and the chat carries on from there, dropping the turns below. The result is a fork: a new conversation that reads the earlier turns from the old one and holds none of the later ones. The original is unchanged, and nothing is copied, since the fork stores only the turn it was cut at.

Draft. A *workflow* *Ruby* has written and you have not yet *run*. Read it, argue with it, change it. Checking that it will run costs nothing, and nothing is spent until you press Run.

The workflow, and fixing it in advance

Workflow. One set of *steps* and the graph joining them. This is the thing you agree, version, date and hand to a commissioner. One *question* may have several, and each may have several *runs*.

- You never number the steps. Each step says what it needs and what it makes, and the order falls out of that. Two lines of work can run side by side and come together later.
- **A name and a version.** The name says which way of answering the *question* this is, and *Ruby* is asked to say how each one differs from the others. A workflow with no name is refused, and so is a name and version already used for different steps. **foo v2** is a different workflow from **foo v1**; the shared name says the two are attempts at one idea rather than unrelated ideas.
- **Change the steps and it is a new workflow**, at the next version. A *run* fails, *Ruby* fixes it, and that is **foo v2**. Running the same steps again, or only up to a named step, is the same workflow at another run.
- **A workflow's state** is one of: not run, running, waiting, failed, candidate, accepted, archived. A *question* stores only whether it has been archived. Whether it is answered is read off its *answers* rather than kept: a question with an accepted *workflow* behind it is answered, every other one is unanswered, and a question dropped on purpose is archived. Nothing anybody sets can say a question is answered while nothing stands as its answer. What the *project hub* shows against it, what to do next, is read from its workflows rather than stored: ask it, read what *Ruby* said, run the *draft*, watch a run, read what came out, or it is answered.

Step. One instruction inside a *workflow*. It says which *results* it reads and what it produces, and it is the smallest thing that can succeed or fail on its own. The kinds:

- **select**, **sample** and **select_passages**: three names for one step, saying what may be read next and producing a *sample*. Over documents, over passages, or both.

- **code**: mark up the text, producing a *coding* and its *quotes*.
- **compute**: work out a *figure*.
- **align**: group two or more codings of the same text, so both the agreement and the silence can be counted. A group records which codings covered the passage and which never marked it.
- **judge**: apply a *rubric* and reach a *judgement*.
- **note**: write it up.
- **synthesise**: make text to *validate* a method on.
- **theory**: hang the counting and the judging on the theory of change a *workflow* declared, and file the annotated theory as a *result* in its own right. It reads the links from the fan-out that tested them, with *links_from*, so the shape is stated once. Where a workflow declares no links at all, it reads them out of a coded table instead, taking each distinct pair of ends as a link and counting the passages behind it: that is a theory of change read out of the material rather than checked against it, and it is what Causal Map has always made. This is the thing a contribution analysis is for, and it is not the margin map.

Fan-out and prong. A *step* written with *for_each* runs once per item in a list, and each of those runs is a prong. Prongs are built from the same inputs and never receive each other's outputs. Usually a later step gathers them into one table, but it need not: a *workflow* can end in a fan-out, with one *judgement* per partner organisation and nothing combining them.

Result. What a *step* produces: a *sample*, a *coding*, a table, a *figure*, a *judgement*, a paragraph. You name the type yourself, in your own words.

- A *source* is never a result. What a sampling step produces is the *sample* it drew; the documents themselves stay Causal Map's.
- A result never changes. Run the step again and the new result sits beside the old one, so the two can be compared.
- Every result knows which results it came from. That chain is its *lineage*.

Result set. Everything a *workflow's runs* produced, finished or not. A run that broke at its fourth *step* still made three things.

Validation. Showing that a *workflow* works at all, by running it over synthetic texts whose *judgements* are known in advance. The *synthesise* step that makes those texts is built; nothing yet compares what the workflow awarded against what was expected. Independent of *registration*: a registered workflow may be useless, and a validated one may never be registered.

Registration. Fixing the method before the evidence is read, with a version, a date and a named person. Anything a commissioner would not want changed after the first results can be registered: the *sample*, the *coding* instruction, the counting *rule*, the *rubric*. What Rubicon registers today is the rubric alone, so a *workflow* that stops short of a *judgement* registers nothing, and a sampling frame revised after a first look is dated in the *workflow* version rather than held by a registration.

Running it

Run. One execution of a *workflow* or part of one. It records order, *results*, model, duration, cost, and every model call with the prompt as sent and the answer as returned.

- A *step* at a time is the intention: run the *coding*, read it, fix the instruction, run it again, and only then go on to the counting.
- *until* stops at a named step. *continue_from* picks up what an earlier run made, recording the inherited steps as skipped.

rerun does named steps again. On the command line, *--until <step>* when starting and *resume <run-id> --rerun <step>* afterwards.

- A run that was told to stop records *cancelled* rather than failed.
- *rerun* names steps, and cannot name a fan-out, whose steps are called *test_link[...]*, one per prong. Name the prong.
- Running twice is repair or a reliability check, so runs are a *workflow's* history rather than things to compare. What tells two runs of one workflow apart is when each ran, how it ended and what it cost. It is never the instructions, and never the *sample*, since both belong to the *workflow* and changing either makes the next version rather than another run.
- **A run is built to repeat, so most of what could vary is held still.** A draw states its seed, and a *step* whose call has been made before inherits the answer under *reuse*, so a second run over an unchanged corpus can finish having spent nothing. What is left free to move is the material, because sampling reads the *project* as it stands and a *source* added since widens the frame, and the models themselves wherever the text has changed or a step has turned reuse off.

Ledger. The record of everything that ran: *runs*, *steps*, *results*, *quotes*, model calls and costs. The page and the command line both read it, and it is the only full account of a run there is. Only the cost of each real call is written a second time, to Causal Map's own *ai_logs*.

Reuse. Answering a *step* from an earlier identical model call rather than paying for it again. The key is the request as sent, source text included, so an edited *source* cannot inherit an old reading.

What comes out

Coding. A pass over the text that finds passages and records whatever you asked about each one, in columns declared in advance. It is what a *code step* produces, and its rows are *quotes*.

- Causal Map has one canonical coding per *project*, because it builds one model of the sources' causal cognitions. Rubicon has no such object. A coding answers the *question* its own instruction asks, so a project holds as many codings as it has questions.
- A coding whose every row was refused by the declared columns is refused, rather than recorded as having found nothing.
- **A coding may cover less than the whole text.** A chunk whose answer comes back unreadable is recorded as unread and the pass carries on, so one reply in two hundred no longer takes the other 213 with it. The count is on the asset beside the segments a model declared empty, so a figure built on this coding can state 23 of 24 chunks rather than implying all of them. Past a share the workflow declares, the step refuses instead: a model that has stopped answering must not produce a table built on a tenth of the text while looking complete.

Quote. A marked passage: the words and their position, checked against the real text and snapped to it where the match is exact, canonical or gapped, and kept as the model wrote it with approximate offsets, labelled as such, where the match is only fuzzy. A quote the machine cannot find in the *source* at all is thrown away rather than stored, because a plausible quotation that appears nowhere is the most dangerous thing an AI can produce. Everything else here exists to stay attached to these.

Figure. One value a *compute step* produced, a number or a table where it grouped, carrying the *unit* it counted across, the filter it applied and what it took from each unit, all stated rather than implied.

Rubric. A table whose rows are the verdicts on offer, each with a shortname, what it stands for, and how to make that judgement. It is how you get from "five out of seven" to "adequate".

- A rubric is not itself a *step*. A *judge step* applies one, reaching a *level* on each *criterion* and then combining them where the rubric says how.
- A shortname alone is not a standard. "Green" says nothing. "Above average for a project of this type" is a claim.
- Where one description fuses several *criteria* and you would rather keep them apart, the column splits into one per criterion. Rubicon takes either.
- The verdicts are whatever you say. Nothing in the app knows which set you will use, or assumes red, amber and green, or three levels.
- Some *workflows* never use one. A *step* may produce a yes, a shortlist, or a short story written to make a typical account palpable, without making a final *evaluative judgement*.

Criterion, level and band. One *judgement* may rest on several criteria: salience, strength, frequency, coverage, whatever the *question* needs. The levels are the verdicts a criterion can take, and two criteria in one *rubric* need not share a set of levels: one can be red, amber and green while the next runs 1 to 5. A band is a contiguous run of levels, such as "adequate or better", and it only means anything where the levels are ranked.

Judgement. One verdict, produced by a *judge step*: what the material earned, why, and which *quotes* it rests on.

- **Where the rubric keeps its criteria apart, it must say how they combine**, because whether one red outweighs two greens belongs to whoever wrote the standard. A rubric that does not say produces a judgement that reports each criterion and states that nothing combined them.
- **Two ways to say how.** Standards test the set: every criterion green, no criterion red. Or *combine: worst*, where the weakest criterion decides, and *combine: best*, where the strongest does. Some claims are chains and a chain is as strong as its weakest link: a theory of change running A to B to C fails at whichever link the evidence does not support, however good the others are, and standards cannot express that. Reports average instead, because the natural summary of a table is a total. *best* is the branch, where two routes reach the same outcome and the better route carries the claim. Nesting them is not supported, so a chain of branches is written as branch summaries combined with *worst*.
- **Combining reads the order the verdicts were declared in, best first.** That is only safe where somebody

declared them on purpose, so a rubric whose verdicts are inferred from its standards is refused, and so is a criterion whose verdict falls outside the declared set. Ranking a verdict nobody declared would turn an unresolved criterion into a confident answer.

- Some verdicts are settled by arithmetic over *figures*, others by the AI reading the evidence against the written description. Both are legitimate, they carry different weight, and the judgement records which was which. A criterion is settled by arithmetic as soon as any of its standards has a *rule*, written *rule: {metric: share, gt: 0.5}*: one figure the input table already holds, then one or more comparisons from *gt*, *gte*, *lt* and *lte*, every one of which must pass. A rule that compares nothing is refused. Where the step reads more than one table, *of:* names which. Standards are tried in the order written and the first whose rule passes wins, so write them strongest first. A standard without a rule is passed over, and an unmatched criterion falls through to the last standard, whether or not that one has a rule.
- **A criterion about the method cannot cite the material, so it is answered without quotations.** Whether enough interviews yielded anything, whether one category swallowed the rest, whether a comparison rests on a big enough base: each is a fact about what the run did rather than about what anybody said, so nothing quotable could evidence it and a judge asked to read for one is refused, correctly, for citing nothing. Two ways to write one. A *rule* over the figures the counting already produced, which is arithmetic and reproducible. Or *evidence_from: run* on the criterion. A model then answers from the run's own record: the workflow as executed, and what each input asset reports about its own making. Keep for the reading model the criteria that genuinely need the words, such as whether the passages returned belong in the category they were given.

Evaluative judgement. A *judgement* that restricts a set of options which are defined and ranked by worth. This is the kind that needs *registration*, and the three conditions are under Worth is declared by a person, below.

Lineage. The chain by which every *result* records which results it came from, ending in *quotes* and the words in a *source*. If the walk back breaks anywhere, nothing else about the *run* counts.

Accepting. Naming one *result* as the *question's* answer. It points at a result rather than at a *run*, so a table can be accepted as readily as a *judgement*. One answer stands at a time, which the database holds as a unique index rather than a careful function, and a question is answered exactly when one does.

Archiving. Hiding *runs*, or a whole *question*, when you are finished with them. A *workflow* shows as archived once every run of it is. It never deletes.

How a piece of work goes

Fully specified means every decision settled before anybody sees the evidence. That is the hardcore end rather than a requirement.

1. State the question, and which part of it this workflow answers.
2. Name the sources.
3. Write the coding instruction.
4. Write the counting rule.
5. Write the rubric.
6. Validate it on synthetic texts.
7. Register it with the commissioner, dated, before the analysis runs.

8. Run it, and report what comes out.

Steps 6 and 7 are independent: a registered workflow may be useless, and a validated one may never be registered.

In practice we iterate. The question turns out not to fit the material, a distinction nobody expected forces new coding columns, a finding rests on one talkative respondent. So every version is dated: change a threshold after seeing the numbers and that shows, change it before and that shows too.

Where the people go. The eight steps above read as though one person writes a workflow and a machine runs it. That is the degenerate case. Most evaluation worth doing has other people in it, and the places they belong are predictable enough to mark in advance:

- **Before step 1**, agreeing what the question actually is, which is usually a negotiation rather than a briefing.
- **At steps 3 to 5**, when somebody who knows the setting reads the coding instruction and the rubric and says which distinctions will not survive contact with the material.
- **After the first run of a coding step**, reading what came back, fixing the instruction, running it again. This is the loop the engine was built for: run one step, look, revise, continue.
- **At substantiation or member checking**, where the people described get to say whether the description is right.
- **At sense-making**, where a group reads the results together and decides what they mean, which is not the same as being told what the machine found.

- **At the verdict**, where a person may override it, and the override sits beside the machine's version rather than replacing it.

A workflow should say which of these it uses and where. Today it cannot: the engine can stop at a named step and continue from it, so the mechanism half exists, and there is nothing that marks a point as one where people are meant to intervene. That is a gap rather than a position.

The tension with registration is real, and we take a side, under **(Pre)-registration** above: registration records ordering rather than commanding fixity, so revise as often as the work needs and never revise a version.

We claim no determinism and no full reproducibility. AI coding varies between runs, and machine agreement with human coders is well short of perfect. What survives is **auditability**: a reader who disagrees can find out exactly where.

What a workflow can express

A workflow is not a checklist of one instruction per question. It is a graph, and the shapes below are the reason the general parts are worth having. Every one of them is stated in the YAML before anything runs, which is what makes the result arguable rather than merely produced.

Fan out, and gather. `for_each` runs a step once per item, written beside `spec` and never inside it, in three forms: a list written down, a field of an earlier step's asset, and every distinct value a source column takes. Twenty QCA cases, twelve process tracing tests or five rival theories are one line rather than twenty hand-copied chains. Each prong carries the item's id on its step id and its outputs, prongs never see each other, and a later step gathers them by naming the role the fan-out declared, with the item arriving as an ordinary column so `over: {group: item}` gives a row per case. A prong that found nothing still appears, as a nought.

Split the corpus, and keep the halves apart. A `select` step decides what may be read, and a run records which sources it actually read, so a workflow can write its tests against one half and score them against the other. Nothing yet refuses to score a test on the material that produced it; the record makes it visible, and the assertion is a gap.

Change the reading rather than the question. `chunk_size` and `segment_quota` on a `code` step decide how much text the model weighs at once and how much it has to find per segment. Halving the chunk finds more of the same kind of thing, which is why two codings are only comparable when both were given the same budget. `align` records exactly that, comparing what the specs said about chunk size, quota and coverage, and putting the disparity in front of the reader rather than judging it.

Test the instrument on a subsample. Run one instruction twice over ten sources and group the results with `align`: the flip rate is run-to-run variation, which is what an auditable claim rests on. The second run needs `reuse: false`, or it inherits the first one's answers and the flip rate comes back at nought for a mechanical reason, which `align` reports as a concern in its own words. Run two differently worded instructions over the same ten and the difference is instruction sensitivity. Neither is a finding about the world, and both belong in the report.

Produce several tables from one coding. `compute` takes the same rows and gives a share per case, a count per actor type, a breakdown by year, and the same figure again restricted to speakers with no stake in the programme. They are separate figures with separate bases rather than one number carrying caveats, and a share over a frame the draw cannot speak for is refused rather than printed.

Tabulate the judgements before rolling them up. Twelve tests judged one at a time against one rubric give twelve verdicts in a table, each carrying its own quotes. The roll-up is a second `judge` against a second rubric, which states in advance how the parts combine: `combine: worst` for a chain that is as strong as its weakest link, `combine: best` where any one route is enough. An average is what a report reaches for when nobody wrote the rule down.

Stop, look, and carry on. `until` stops at a named step, `continue_from` picks up what an earlier run made and records the inherited steps as skipped, and `rerun` does one step again. That is the loop a person actually works in: run the coding, read what came back, fix the instruction, run it again, and only then go on to the counting.

The rubric is the part all of this exists to serve. Everything above is description, and description has a backstop, since the passages are still there when a count is wrong. A judgement of worth has none, so the rubric is the artefact that gets a name, a version, a date, a person, and is never rewritten in place, with every run recording which version was in force and whether it was frozen before the evidence was read.

Named methods are recipes, and a method's need becomes a dial

No named method reaches a code path. Process tracing, contribution analysis, realist evaluation and outcome harvesting are recipes written in the general parts, and Ruby offers one when a question suits it. A tool built round a fixed list is wrong the first time somebody wants something slightly different, which is most of the time, and a method encoded in software is a method nobody can argue with.

The recipes, in one line each.

- **Process tracing.** Rival published theories against a corpus, using the four evidence tests (Ricks & Liu 2018), with the tests written before the material is read and a synthetic set used to check they can tell the theories apart at all.
- **Realist evaluation.** No theory to start with, generating candidate mechanisms, then measuring how many of them a competent reader could have written without doing any fieldwork.

- **Contribution analysis.** The theory of change the programme committed to, each link tested, and the rival explanations for the whole outcome taken as seriously as the programme's own account. The product is an annotated theory of change rather than a verdict.
- **Outcome harvesting.** Mostly a conversation between people, and it should stay one (Wilson-Grau 2018). Two analysis questions in the middle are worth a machine: whether the outcomes form a pattern of progress against the objectives, and which objectives have nothing against them.

Where a method wants something that would be useful more widely, it becomes a general part rather than a special case. Outcome harvesting needs the thing being counted to be a person rather than a document, and so does anything else about actors, so that is a dial. The test is whether a second method would use it, and the ones that pass keep arriving from three directions at once.

Principles

Worth is declared by a person

Julian King put the objection in October 2025: AI should not be making evaluative judgements about human affairs. Quirkos and SenseMaker refuse AI analysis altogether, and f4analyse will not automate analysis of a whole corpus. The evidence-synthesis bodies draw the line in the same place, at the point where a model makes or suggests a judgement (Fleming & Noel-Storr 2025), and the case that the technology strengthens rather than threatens qualitative inquiry is being made alongside it (Wise et al. 2026). We agree.

- Nothing inside a scale makes it evaluative. Green is not better than red because of anything about green, and no amount of looking at the material tells you where adequate begins.
- A descriptive finding has a backstop: count the households wrong and the passages are still there. An evaluative claim has none. A standard set after seeing the numbers can be made to say anything and stay consistent with all of them.
- So the standard has to come first, from somebody who put their name to it. What registration covers, and why it stays an option, is under **(Pre)-registration** above.
- A rubric is versioned and never replaced, and carries who registered it and why, though on the run path both are written by the machine rather than by a person. Freezing a version, and recording whether the freeze came before the run, is designed and not yet built.
- The argument is with tools that ask a model to decide what good looks like and then present the answer as a finding.

A step is making an evaluative judgement when all three of these hold. The app does not test the three. It uses the structural proxy, a step of type **judge**, and that is where it requires a rubric.

- It **restricts** a set of options declared in advance, usually to one of them, sometimes to a band such as "adequate or better".
- Each option **says what it takes to earn it**. "Green: every partner described a change they could date and name a cause for" is a standard, because you can argue about whether the material meets it. "Green" on its own is not, because two people reading it would not agree what it means and neither would a model. A coding column whose levels are bare names

is refused outright; a rubric that lists its verdicts must say what each stands for, and one that never lists them runs with the meanings reported as missing.

- The options are **graded by worth**. Health, farming and income is classification, because it ranks nothing. A frequency table over speculative, reported and evidenced has options and grading, but it reports the distribution rather than choosing a point on it.

Grading is what makes a band coherent, so **at_least** works on an ordinal and never on a category. The test applies to the answer a step gives, so a workflow is usually descriptive in the middle and evaluative at the end. Structurally: whether it ends in a judge step.

Transparency is not enough on its own. There has to be a place to disagree: override the verdict, rewrite the rationale, reject a cited quote, mark a criterion unanswerable. The override would sit beside the machine's verdict so the difference is itself a finding, carry who and when, and be what everything downstream used. None of it is built. Today the only human decision the app records is accepting or withdrawing a whole result.

Every claim points at a quote

- Any narrative or verdict about the material must lead back to the quotes it rests on (Moravcsik 2014). The system refuses to store a note or a judgement that cites none, and it gets there by requiring the coding to be a declared input of the step making the claim, so a claim cannot drift away from its evidence. A table is not held to this.
- This is the likeliest way a tool like this fails, and it fails without showing: two steps from the source the prose still reads well and the numbers still look precise, while nothing connects either to a document. Note it's easy for an AI tool to reach a conclusion and then search the text for quotes to back that up. Rubicon does not fall into that trap.
- **Some findings are about the method rather than the material:** was contrary evidence looked for, was the sample drawn in a way that supports the claim. No quotation can evidence the absence of a search, so those cite the run record instead, and the judgement says which kind each verdict was.
- A criterion should be one kind or the other, and nothing refuses one that is both. "Was anything found that cuts against the hypothesis, and how was it handled?" is two questions, and asked as one it makes a model rate the method by reading the evidence.

- A band should stand for one state of affairs, and this one is on you: nothing reads a band's description. Where the worst band covers both "contrary evidence was found and it is damaging" and "nobody looked", the reader cannot tell which happened.

Anything that can legitimately find nothing must show that it at least looked, and a negative finding cites the search rather than a quotation. Both are under **Being aware of absences** above. What the machinery reports, beside what it found, is the passages dropped because the quote could not be located, how much of each document was examined, and how many model calls there were and what they cost.

Build the thing, and not only the verdict

A verdict is one word standing for eight links, three hundred passages and nineteen documents, and tracing back through it does not rebuild what was compressed. So where you would expect a graph, a table of outcomes or a cross-tab to be in front of you, the workflow builds it and declares it as a result, which usually costs nothing because it was a step on the way anyway. Such a workflow can stop there: a table can be accepted as the answer as readily as a verdict. The mirror fault costs more, so a question that wants a number should not pay for a graph nobody asked for.

Say what you are comparing against

A programme did well: compared with what? Seven answers, and they are not interchangeable.

- **A denominator.** Eleven of the fourteen partners. Needs a unit and a stated base.
- **An alternative.** Which of two readings the material better supports. No denominator at all.
- **A written standard.** A rubric agreed in advance. The only one that works when there is nothing to compare with.
- **A comparison case.** The programme before it started, a similar one elsewhere, a site that got no support. Rests on the cases being alike in the ways that matter.
- **What was expected.** The theory of change, the target, the plan. Cheap, and it measures delivery rather than worth.
- **What would have happened anyway.** Usually the hardest. Approachable through causal mapping and other generative accounts of causation, which treat people as causality detectors whose judgements carry counterfactual implications.
- **The evidence itself.** Sometimes the finding is that nothing here is well enough evidenced for any of the above.

Most confusion in evaluation reporting comes from sliding between them: counting how many people said something, then writing as though that settled whether it was any good. Two obligations follow, both stated above: **search both sides equally**, under Avoiding bias by overfitting to the data, and **state the base, including the documents that gave nothing**, under Being aware of absences.

Where this sits in the literature

- Scriven's logic of evaluation separates description, "What's so?", from valuing, "So what?", then reaches a verdict through criteria of merit, standards of merit and a synthesis into overall worth (Scriven 1980, 1991; Fournier 1995).
- Davidson puts it as a test: an evaluation must ask not only "What were the results?" but "How good were the results?" Her worked failure is an executive summary of counts and percentages with no verdict in it (Davidson 2014).

Searching both sides in one pass over the same text is a convention rather than something the app enforces. What the app does is check, where two codings are compared, that they had the same opportunity: same sources, same settings, comparable segment coverage, and no shared reused answers. It reports the disparity on the comparison and refuses nothing.

What it refuses, and what stays soft

- **"Read the documents and tell me whether the project was effective."** Answered in one pass, that produces the thing this app exists to replace: a confident verdict, no stated standard, and no way for anyone to disagree with it in particular.
- **"Is this a good project?"** is four questions in one coat: good for whom, compared with what, against whose standard, and how much of that this material can answer.
- The test is whether the question is well defined. Nothing is refused for being about tone or feelings or anything else that sounds soft.
- **Refusing is rarer than it sounds.** "What percentage of the budget went on training?" is well defined where a report states the figure and unanswerable where none does, which is a fact about the corpus. The answer is usually a workflow that goes and looks. If nobody wrote the figure down, that is the finding, with the search behind it.
- **Some questions are well defined in parts and not as a whole.** "What share of the fellows who wanted a policy meeting got one?" is two codings and a ratio. A single pass would be arithmetic done in a model's head.
- **A question whose answer is in a spreadsheet is refused, and that is the point.** Hours between landfall and the first cash payment, cost per person reached: those live in a payments system with no words behind them. Everything here rests on a number being followable back to the words somebody said, and mixing the two makes every number only as traceable as the weakest. The workflow says which part of the question it answers, and you bring the other figures in when you write.
- **A number spoken in a document is in scope.** "We reached about four hundred children", said by a school director, has words behind it, a speaker and a position in a source. A coding step declares a numeric column and the counting does arithmetic on those.
- **A judgement may refuse the question.** Where the material shows the question rests on something that did not happen, or compares two things the sources cannot compare fairly, the judge says so instead of awarding a band, and cites what that rests on. Awarding a band is worse, because the band gets quoted and the doubt does not. One criterion challenging the question sinks the overall verdict rather than being outvoted.

Open questions

An item leaves when it is decided: built and durable it graduates into the principles above, abandoned it goes without ceremony.

- **Validating a workflow on synthetic contrasting texts.** The `synthesise` step is built, with its refusals: it will not generate from the rubric alone, it will not take a set with no decoys unless you say why, it demands an expected verdict on every document, it will not default the generating model, and it namespaces what it makes as `syn`: so it cannot be mistaken for a real source. What is missing is the last step, the one that compares the verdicts awarded against the verdicts expected and reports the separation as a figure. Design in `rubicon/validate a workflow on synthetic contrasting texts xkTODO xpSTEVE.md`.
 - What the texts have to be, and why they cannot come from the rubric alone, is under **Validation with synthetic material** above. The refusals in the step enforce it.
 - Form: a `synthesise` step, then the ordinary workflow run over what it produced, never a path of its own. Generated documents stored as assets and discrimination reported as a figure.
- **Near-miss reuse.** Identical reuse is settled by hashing the request. What an evaluator meets is a theory of change very like one this project already holds, where rebuilding costs an hour and ten dollars. Two things would make it safe, neither automatic: the difference stated in terms somebody can judge, which links differ and which clause changed, rather than as a similarity score; and the reuse agreed rather than assumed.
- **Reading an accepted answer rather than a raw asset.** `from_run` reaches an asset, which is the raw output of a step and not what anybody agreed to. What a person accepted is an answer: the row carrying the asset, who elevated it, when and why. So a comparison can be built on a run that was never accepted, or one whose acceptance was later withdrawn, and nothing in it says so. An input of the shape `{answer_to: <question id>}` would resolve through the answers instead, refused where the question has none rather than falling back to whatever ran last. The copy would carry everything except the standing: the asset, its lineage, its quotes, the rubric version in force, and the earlier elevation as history, with the accepted status downgraded, so nothing downstream can present it as still standing. Design in `rubicon/xkTODO a comparison should read an accepted answer xpCLAUDE.md`.
- **Building a theory of change, rather than checking one.** Checking is tractable, since each link is a proposition and asking the material what supports it is close to what the app does now. Building is harder: the object is assembled from hundreds of fragments and no fragment contains it, so the categories cannot all be declared in advance and a source read late can change what an earlier passage meant. Causal Map does this well, so the question is what an auditable version looks like when the structure emerges rather than arriving.
- **Running a step at a time from the page.** The engine has `until`, `continue_from` and `rerun`, and the command line reaches them. The page posts a draft and gets back the whole workflow, so the way this app is meant to be used is the way nobody using it can work. The per-step blocks of the single pane are where the control will hang. One naming trap waits there: `runs.status` already uses `partial` for a run whose steps failed, so the workflow-level word has to be a different one.

- **Whether the sample belongs to the workflow or to the run.** Run over twelve sources, then over all fifty-three: intuitively the same workflow twice. The engine disagrees, because the sample lives in a step's `spec` and one id-and-version pair cannot cover two different specs. Unsettled, and the pane needs a run picker whichever way it goes.
- **Forking a workflow from a step,** discarding everything downstream. That makes a new workflow. Not built.
- **A place to disagree.** Overriding a verdict, rewriting a rationale, rejecting a cited quote, recorded beside the machine's version and carried downstream. The principle is above and the surface does not exist.
- **Composite markup.** Assembling one object from evidence scattered across passages, sources and speakers, with every part keeping its own trail. A realist configuration is context in one passage, reasoning in another, an outcome in a third. Today those come together inside a model's rationale, which reads well and cannot be queried. Wanted by more of the work than any other open item.
- **Comparison against what was expected.** A workflow can compare two codings and compute a proportion. It cannot yet read a written expectation and ask the material whether it happened, which would report which links nobody mentioned, something an evaluator cannot get any other way.
- **The draft-to-run link.** A run stores a copy of the workflow it ran rather than the id it came from, so the ledger cannot say which proposal an attempt came from. Two things guess at it by workflow id and version: `read_failure`, reporting what a failed run was estimated to cost, and the page's `workflowsFor`. Measured against the live ledger on 6 September 2026, the guess is unsound at a rate that matters. Fifteen of 64 question-and-version keys carry more than one draft, most of them with different estimates, so an attempt can be reported at a figure from a draft it did not come from. One column, written when the page posts the draft's id, would close it. Matching the run's stored spec against a draft's parsed YAML is the cheaper half-measure: it named exactly one draft for 36 of the last 60 runs and was never ambiguous, though it found nothing at all for the other 24.
- **Project home shows raw column names,** fifteen lines of `s_#Name of province` and the rest, where the reader should see the name a person would use.
- **Flagging a background document has no button.** You type the value into a custom column on the source.
- **Arranging the canvas by asking.** "Show me the quotes, longest first", "put the two codings side by side". Ruby proposes workflows and has no tool that touches the canvas, so today the page shows what it shows. Not built.

Not settled at all:

- Making the unit a person or an organisation rather than a document, when who they are is only discovered during the coding.
- Expressing a base that is not a set of documents, such as "the forty partner organisations", when only twelve appear in the material.
- Pausing a workflow for people to do something in the world, then picking up where it left off.
- Merging results that turn out to describe the same thing, keeping the trail back to every passage that contributed.
- Naming.

References

- Apgar, Bradburn, Rohrbach, Wingender, Cubillos Rodriguez, & Arias (2024). *Rethinking Rigour to Embrace Complexity in Peacebuilding Evaluation*. Evaluation, 13563890241232405. <https://doi.org/10.1177/13563890241232405>.
- Apgar, & Aston (2025). *How Do We Define and Support Quality and Rigor in Causal Pathways Evaluation?*.
- Aston, & Apgar (2022). *The Art and Craft of Bricolage in Evaluation*, CDI Practice Paper 24. <https://doi.org/10.19088/IDS.2022.068>.
- Flemyng, & {Noel-Storr (2025). *Position Statement on Artificial Intelligence (AI) Use in Evidence Synthesis across Cochrane, the Campbell Collaboration, JBI and the Collaboration for Environmental Evidence 2025*. Cochrane Database of Systematic Reviews. <https://doi.org/10.1002/14651858.ED000178>.
- Moravcsik (2014). *Transparency: The Revolution in Qualitative Research*. APSCC Newsl., 47, 48--53. <https://doi.org/10.1017/S1049096513001789>.
- Reitman (1964). *Heuristic Decision Procedures, Open Constraints, and the Structure of Ill-Defined Problems*. In *Human Judgments and Optimality*.
- Ricks, & Liu (2018). *Process-Tracing Research Designs: A Practical Guide*. PS - Political Science and Politics, 51, 842--846. <https://doi.org/10.1017/S1049096518000975>.
- Simon (1973). *The Structure of Ill Structured Problems*. Artificial Intelligence, 4, 181--201. [https://doi.org/10.1016/0004-3702\(73\)90011-8](https://doi.org/10.1016/0004-3702(73)90011-8).
- Wise, Gresalfi, & {Spencer-Smith (2026). *Why AI Is Not the Enemy: Opportunities to Strengthen Core Commitments of Qualitative Inquiry Through Trustworthy AI-in-the-Loop Analysis*. International Journal of Qualitative Methods, 25, 16094069261435579. <https://doi.org/10.1177/16094069261435579>.
- {Wilson-Grau (2018). *Outcome Harvesting: Principles, Steps, and Evaluation Applications*. IAP.